Д. Серікбаев атындағы Шығыс Қазақстан техникалық университеті

«DevOps құралдары» пәні

ЗЕРТХАНАЛЫҚ ЖҰМЫС № 2, 1-бөлім

Dockerfile және құрастыру кэші

Орындаған: Есімханова Аружан Түсіпханқызы

Тобы: 25-ИСТК-2о

Оқу жылы: 2025-2026

Өскемен - 2026

МАЗМҰНЫ

1 Жұмыстың мақсаты 3

2 Теориялық мәліметтер 3

3 Жұмыстың орындалу барысы 4

4 Жұмыс нәтижелері 11

5 Талдау 11

6 Қорытынды 11

7 Пайдаланылған әдебиеттер тізімі 12

1 Жұмыстың мақсаты

DemoApi үшін көп кезеңді Docker бейнесін құрып, тәуелділіктер кэшінің қайта пайдаланылуын тексеру.

2 Теориялық мәліметтер

Docker бейнесі файлдық қабаттардан құралады. Көп кезеңді құрастыру SDK мен аралық нәтижелерді соңғы орындалу бейнесінен бөледі. Тәуелділіктерді бастапқы кодтан бұрын көшіру restore қабатын қайта пайдалануға мүмкіндік береді.

3 Жұмыстың орындалу барысы

DEVOPS · 2-БАҚЫЛАУ НҮКТЕСІ · 3-АПТА

3.1 1-бөлім: 1-5-тармақтар. 03-материал.

Орындаған: Есімханова Аружан Түсіпханқызы
Тобы: 25-ИСТК-2о
Күні: 2026 жылғы 22 қазан

3.2 Жұмыстың мақсаты

ASP.NET Core / .NET 10 негізіндегі DemoApi сервисін екі кезеңді құрастыру арқылы Linux бейнесіне жинақтау, артық файлдарды алып тастау және бастапқы код өзгергеннен кейін NuGet қабатының қайта пайдаланылуын растау.

1-кесте - Орындау деректері мен тексеру нәтижелері

Орта	Нақты мәні
Docker Client / Server	29.8.0 / 29.8.0
Орындау ортасы	Docker Desktop, Linux x86_64, WSL 2 қосылған
Docker ресурстары	12 CPU, 7.62 GiB жад (кемінде 4 GB талап етіледі)
PowerShell	7.6.5
Негізгі бейнелер	dotnet/sdk:10.0 және dotnet/aspnet:10.0

Ашық репозиторий және барлық бастапқы код

3.3 Dockerfile: екі кезең

1-листинг. Репозиторий түбіріндегі толық Dockerfile. Алдымен тәуелділіктер, содан кейін бастапқы код көшіріледі; соңғы бейнеге publish нәтижесі қосылады.

1-листинг - Код немесе сақталған команда шығысы

```
# syntax=docker/dockerfile:1

FROM mcr.microsoft.com/dotnet/sdk:10.0 AS build
WORKDIR /src
COPY src/DemoApi/DemoApi.csproj src/DemoApi/
```

```
RUN dotnet restore src/DemoApi/DemoApi.csproj
COPY src/ src/
RUN dotnet publish src/DemoApi/DemoApi.csproj -c Release -o /app --no-restore

FROM mcr.microsoft.com/dotnet/aspnet:10.0 AS runtime
RUN apt-get update \
    && apt-get install -y --no-install-recommends curl \
    && rm -rf /var/lib/apt/lists/*
WORKDIR /app
COPY --from=build /app .
ARG APP_VERSION=dev
ENV APP_VERSION=$APP_VERSION
USER $APP_UID
EXPOSE 8080
HEALTHCHECK --interval=10s --timeout=3s --start-period=15s --retries=3 \
    CMD curl -fsS http://localhost:8080/health/live || exit 1
ENTRYPOINT ["dotnet", "DemoApi.dll"]
```

build кезеңі SDK пайдаланып, Release режимінде restore/publish орындайды. runtime кезеңі ASP.NET орындалу ортасын, жарияланған қолданбаны және жұмыс күйін тексеруге арналған curl құралын қамтиды. curl орнату мен apt тазалау бір қабатта біріктірілген.

USER $APP_UID сервисті app пайдаланушысы атынан іске қосады. EXPOSE ішкі 8080 портын құжаттайды. HEALTHCHECK /health/live маршрутын шақырады. ARG құрастырудың ашық нұсқасын береді, ENV оны GET / арқылы қолжетімді етеді. ENTRYPOINT dotnet-ті негізгі процесс ретінде іске қосады.

GitHub-тағы Dockerfile

3.4 Алып тасталатын файлдар және негізгі бейне

2-листинг. Толық .dockerignore. Нұсқаулықтағы үлгіге қосымша есептер, дәлелдер, құжаттама, Postman және ішкі каталогтардағы орта / сертификат файлдары алынып тасталған.

2-листинг - Код немесе сақталған команда шығысы

```
**/bin/
**/obj/
**/TestResults/
.git
.github
.vs
.vscode
.idea
tests/
monitoring/
```

```
scripts/
db/
docs/
reports/
evidence/
postman/
*.md
Dockerfile
compose*.yaml
compose*.yml
.env
.env.*
**/.env
**/.env.*
**/*.pfx
**/*.p12
```

bin/obj ішінде Linux құрастыруына кірмеуі тиіс жергілікті Windows артефактілері бар. Қолданбаны жариялау үшін Git, IDE, тестілер және есептер қажет емес. .env және сертификаттар уақытша қабаттарға да түспеуі үшін құрастыру контекстінен алынып тасталады.

Дайындық тексерулері: docker version Client және Server деректерін көрсетті; hello-world сәтті іске қосылды. Негізгі runtime бөлек жүктеліп, оның қабаттары мен өлшемдері image history және image ls командаларымен тіркелді. Docker build --check ескертусіз аяқталды.

Дайындық пен тексерудің толық журналдары

3.5 Құрастыру және кэшті қайта пайдалану

Алдымен 1.0 нұсқасы құрастырылды. Кейін TaskService.cs ішінде тек тапсырма жасау хабарламасы өзгертілді; жоба файлы мен пакеттер тізімі өзгерген жоқ. Екінші құрастыруға 1.1 тегі және APP_VERSION=lab02-1.1 мәні берілді.

3-листинг - Код немесе сақталған команда шығысы

```
docker build --progress=plain -t devops-labs-demo-api:lab02-1.0 .
docker build --progress=plain --build-arg APP_VERSION=lab02-1.1 `
    -t devops-labs-demo-api:lab02-1.1 .
```

3.6 Хабарламаның өзгерісі (соңғы кодта сақталған):

4-листинг - Код немесе сақталған команда шығысы

```
Создана задача {TaskId} в проекте {ProjectId}
Создана новая задача {TaskId} в проекте {ProjectId}
```

3.7 3-листинг. Екінші docker build командасының нақты үзіндісі:

5-листинг - Код немесе сақталған команда шығысы

```
#12 [build 3/6] COPY src/DemoApi/DemoApi.csproj src/DemoApi/
#12 CACHED
#13 [build 4/6] RUN dotnet restore src/DemoApi/DemoApi.csproj
#13 CACHED
#14 [build 5/6] COPY src/ src/
#14 DONE 0.0s
#15 [build 6/6] RUN dotnet publish src/DemoApi/DemoApi.csproj -c Release -o /app --no
-restore
#15 3.234   DemoApi -> /src/src/DemoApi/bin/Release/net10.0/DemoApi.dll
#15 3.289   DemoApi -> /app/
#15 DONE 3.4s
```

Жоба файлы мен restore нәтижесі кэштен алынды; бастапқы кодты көшіру және publish қайта орындалды. Демек, кодтың өзгеруі NuGet пакеттерін қайта жүктеуді қажет етпеді. Өзгеріске дейінгі және кейінгі SHA-256, сондай-ақ қадамдардың тексеру нәтижелері cache-check.json файлында сақталған.

2-кесте - Орындау деректері мен тексеру нәтижелері

Артефакт	Нақты мәні
Соңғы бейне	devops-labs-demo-api:lab02-1.1
Бейне өлшемі (image inspect)	354,253,942 байт (337.8 MiB)
Платформа / пайдаланушы	linux/amd64, UID 1654

3.8 Құрастырылған қолданбаны тексеру

Бұл тексерулер міндетті 1-5-тармақтарды толықтырады. API нақты Linux контейнерінде localhost:8089 → 8080 арқылы іске қосылды. Тексеру үшін басқа сервистер мен контейнерлер пайдаланылмады.

3-кесте - Орындау деректері мен тексеру нәтижелері

Тексеру	Нәтиже
Контейнердің жұмыс күйі	healthy; /health/live контейнер ішінен орындалады
Қолданба нұсқасы	GET / → 200, version=lab02-1.1
Процесс пайдаланушысы	uid=1654(app) gid=1654(app) groups=1654(app)
Соңғы бейнедегі SDK	dotnet --list-sdks: бос шығыс
Бейнедегі бастапқы код және тестілер	/app ішінде C# бастапқы кодтары, csproj және шешім файлы жоқ
Live және ready	Екі маршрут та → 200
Production ортасындағы Swagger	GET /swagger/index.html → 404
InMemory қайта іске қосылуы	Сол контейнерде: stop/start алдында 1 жоба, кейін 0
Қайта құрастырмай Swagger қосу	Сол бейне Development ортасында: /swagger/index.html → 200
Development ортасындағы OpenAPI	GET /openapi/v1.json → 200, stats бар
Оқу ақаулары өшірілген	Chaos__Enabled=false, /api/chaos/error → 404
API HTTP сценарийлері	26 сұрау: 23 сценарий және тестілік жобаларды жоюға арналған 3 сұрау
Postman CLI коллекциясы	24 сұрау, 73 тексеру, 0 қате; аяқталу коды 0
.NET Release регрессиялық тексеруі	28/28 тест, 0 қате, 0 өткізіп жіберілген; InMemory

3.9 GET / жауабы:

6-листинг - Код немесе сақталған команда шығысы

```
{
  "service": "DemoApi",
  "version": "lab02-1.1",
  "host": "e7ee83006c4d"
}
```

Толық сұраулар мен жауаптар, контейнер журналдары, жұмыс күйі және Postman нәтижелері evidence/lab02 каталогында сақталған. API жергілікті тексерілді; нәтижелер Postman бұлтына жарияланбаған.

3.10 Сұрақтарға жауап және қорытынды

3.11 Контейнер неге виртуалды машинадан жылдам іске қосылады?

Бөлек ядро мен толық операциялық жүйені жүктеу қажет емес. WSL 2 ішінде Linux ядросы жұмыс істеп тұр; Docker namespaces/cgroups жасайды, қабаттарды тіркейді және қолданба процесін іске қосады.

3.12 Неге алдымен csproj, содан кейін код көшіріледі?

Тәуелділіктер тізімі C# файлдарына қарағанда сирек өзгереді. Код өзгергеннен кейін restore-дың бөлек қабаты кэште қалады. Бұл екінші құрастырумен расталған.

3.13 Көп кезеңді құрастырудан кейін соңғы бейнеге не кіреді?

ASP.NET орындалу ортасы, жарияланған DLL файлдары мен тәуелділіктер, конфигурация және curl. SDK, бастапқы кодтар мен аралық obj файлдары build кезеңінде қалады. SDK жоқтығы dotnet --list-sdks арқылы тексерілген.

3.14 Контейнерде Swagger неге жоқ? Оны қалай қосуға болады?

Әдепкі орта - Production; Program.cs Swagger-ді тек Development ортасында тіркейді. Сол бейнеден ASPNETCORE_ENVIRONMENT=Development мәнімен жасалған жаңа контейнер Swagger-ді қайта құрастырусыз ашты. Chaos__Enabled=false оқу ақауларын өшіреді.

3.15 docker stop/start орындалғаннан кейін жобаларға не болады?

InMemory процесс жедел жадында сақталады. Процесс аяқталғанда контейнер сол күйі қалса да, жобалар жоғалады. Нақты тексеруде жоба жасалып, контейнер тоқтатылып / іске қосылды, содан кейін бос тізім алынды.

3.16 Қорытынды

DemoApi көп кезеңді Dockerfile арқылы Linux бейнесіне құрастырылды. Бастапқы код өзгергеннен кейін restore кэші қайта қолданылып, publish кезеңі жаңадан орындалды. Контейнердің healthcheck және API сұраулары тексерілді.

3.17 Дереккөздер және материалдар

Шарттар: жүктелген Devops (1).zip мұрағатындағы 03-материал, «Неделя 3 - Контрольная точка 2», 2-зертханалық жұмыстың 1-бөлімі және 2-дәріс.

Docker: кэш ережелері · Microsoft: ASP.NET Core бейнелері

Репозиторийдегі нұсқаулар мен жауаптар · Сақталған дәлелдер

4 Жұмыс нәтижелері

Linux контейнері, API, hello-world және restore кезеңіндегі CACHED нәтижесі сақталған. Контейнер қайта іске қосылғанда InMemory деректерінің жоғалуы нақты тексерілген.

Тест нәтижелері мен журналдар жұмыс материалдарында [1], іске қосу командалары нұсқаулықта [2] берілген.

5 Талдау

csproj файлы бастапқы кодтан бұрын көшірілгендіктен, код өзгерісі NuGet restore қабатын жарамсыз етпеді. Соңғы бейнеге SDK емес, жарияланған қолданба мен runtime ғана өтті.

6 Қорытынды

Көп кезеңді Docker бейнесі құрастырылып, кэштің қайта қолданылуы тексерілді. Соңғы контейнерде API мен healthcheck жұмыс істеді.

7 Пайдаланылған әдебиеттер тізімі

[1] DemoApi оқу репозиторийі. Нақты тексерулер мен сақталған дәлелдер. Электрондық ресурс. https://github.com/Aruzhaaan-06/devops-labs/blob/main/evidence/lab02/README.md. Қаралған күні: 25.10.2026.

[2] DemoApi оқу репозиторийі. Жұмыстың орындалу тәртібі. Электрондық ресурс. https://github.com/Aruzhaaan-06/devops-labs/blob/main/docs/lab02.md. Қаралған күні: 25.10.2026.

[3] Docker: кэш ережелері. Электрондық ресурс. https://docs.docker.com/build/cache/invalidation/. Қаралған күні: 25.10.2026.

[4] Microsoft: ASP.NET Core бейнелері. Электрондық ресурс. https://learn.microsoft.com/en-us/aspnet/core/host-and-deploy/docker/building-net-docker-images?view=aspnetcore-10.0. Қаралған күні: 25.10.2026.

DevOps пәнінің оқу материалдары: Devops (1).zip мұрағаты, зертханалық тапсырмалар және бақылау нүктелерінің шарттары.

ГОСТ 7.32-2017. Ғылыми-зерттеу жұмысы туралы есеп. Құрылымы және рәсімдеу ережелері. 6-бөлім.